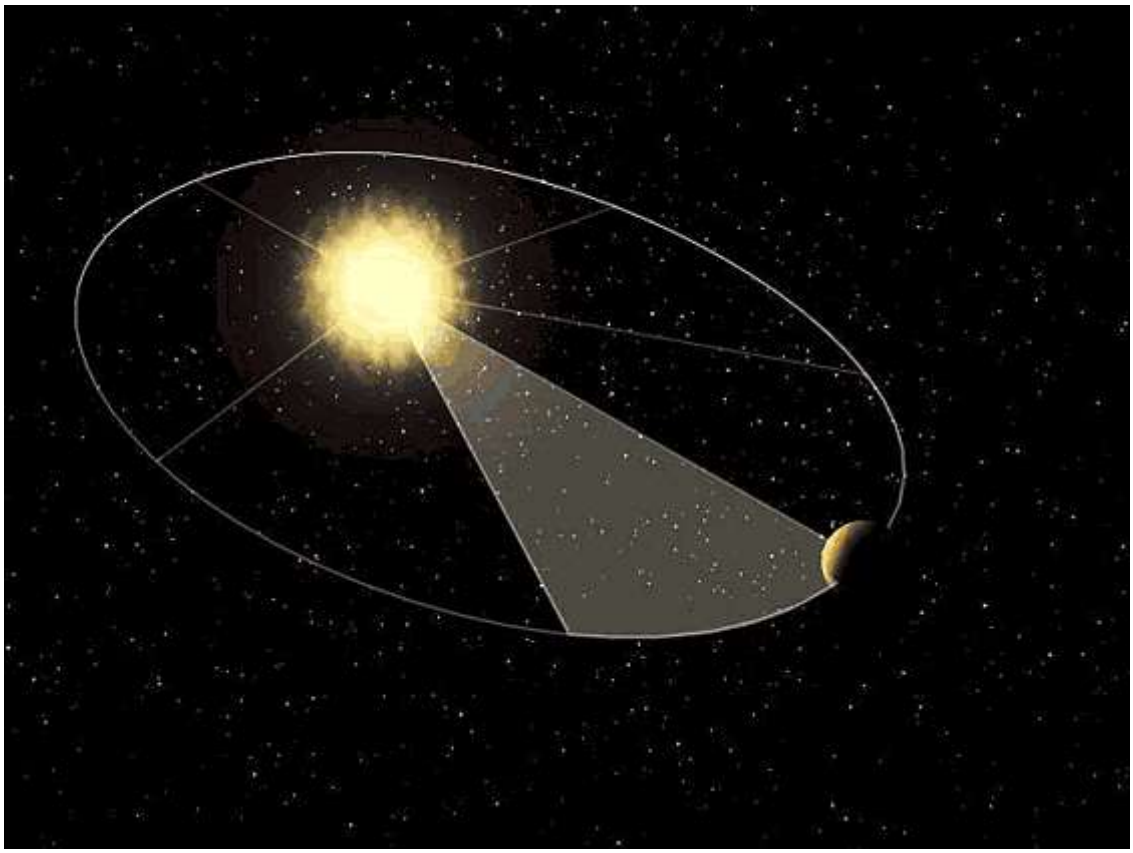
Open in app ↗

Data Analysis in Astronomy: Kepler's Second Law of Planetary Motion using Python

5 min read · Jul 14, 2025

Kritin Murkoth

Listen Share More

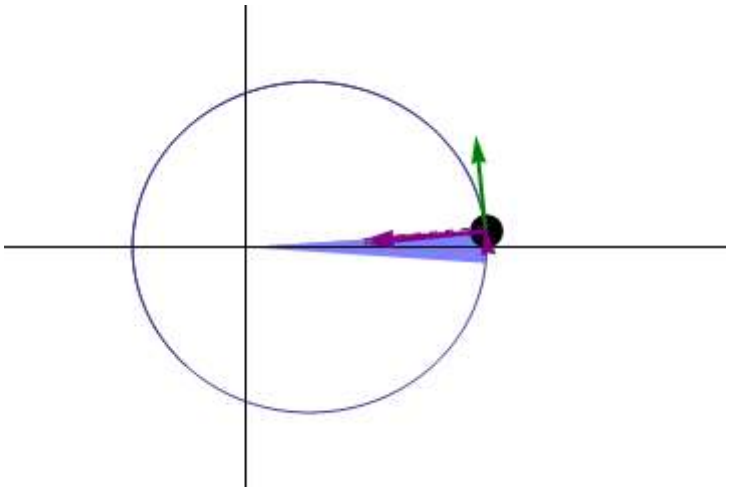


Source: Simply Science

The Kepler Laws' of Planetary Motion form an underlying platform, which has served as the basis for housing subsequent astronomical theories and postulates. It has given astronomers an insight into the further understanding of the planet orbital dynamics and has further strengthened and spotlighted the heliocentric model of planets.

Upon proceeding to the Second Law, its conceptualization can be understood through the following description, as stated by NASA:

> *A **Planet** covers the **same area of Space** in the **same amount of Time**, no matter where it is in its **Orbit**.*



A mathematical visualization of Kepler's Second Law. The purple region represents the area traversed by the radius vector, and the green arrow refers to the instantaneous velocity of the planet. The two purple arrows represent the radial acceleration component and the transverse acceleration component (i.e. the arrow perpendicular to the velocity vector). [Source: Wikipedia]

Implementing Kepler's Second Law using Python

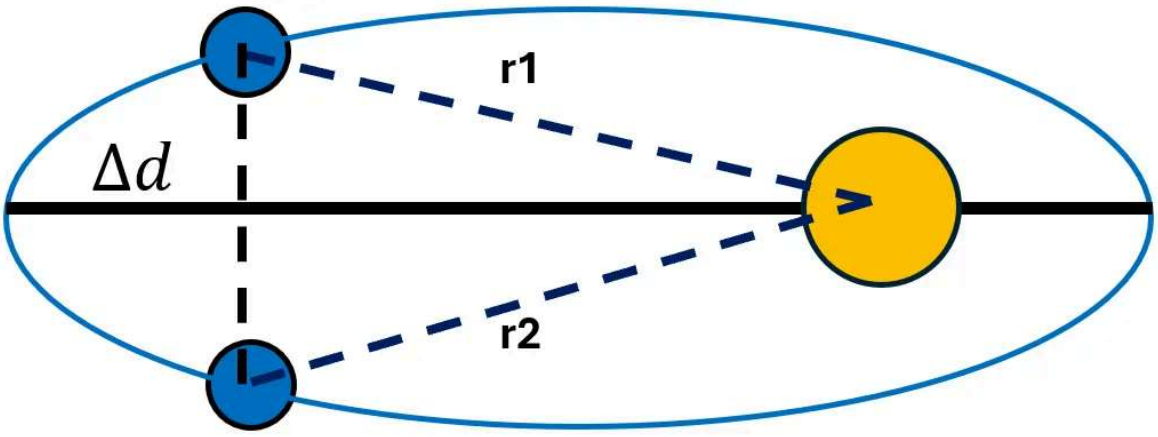
Before proceeding to the code, the following assumptions are made:

· Data used for the computation of celestial mechanics is extracted from the **NASA SPICE toolkit**

· A period of **10,000 delta seconds** has been chosen due to the insufficient data of the kernel being imported

· Ecliptic Reference Frame chosen: **ECLIPJ2000**

· Area computation is done by considering the **area swept out by a triangle**, thus ignoring the area of the semi-circle, having radius (delta_d).



1. Importing the Python Libraries and loading the SPICE kernels

```
import numpy as np
import spiceypy
from matplotlib import pyplot as pyplt
import datetime

spiceypy.furnsh('../Kernels/spk/de432s.bsp')
spiceypy.furnsh('../Kernels/lsk/naif0012.tls.txt')
spiceypy.furnsh('../Kernels/pck/pck00010.tpc')
```

2. Creating the Ephemeris Time Array

```
init_time = datetime.datetime(year=2000, month=1, day=1, hour=00, minute=00,
                              second=00)

delta_sec = 10000     # Time interval

end_time = init_time + datetime.timedelta(delta_sec)

init_time_str = init_time.strftime("%Y-%m-%dT%H:%M:%S")  # Gregorian format
end_time_str = end_time.strftime("%Y-%m-%dT%H:%M:%S")

init_time_et = spiceypy.utc2et(init_time_str)
end_time_et = spiceypy.utc2et(end_time_str)

time_array_in_et = np.arange(init_time_et, end_time_et, delta_sec) #ARRAY
```

3. Obtaining the Area swept by the Earth in particular time frames

```
position_array = []

for et_instnce in time_array_in_et:
    pos, _ = spiceypy.spkgeo(targ=10, et=et_instnce, ref="ECLIPJ2000", obs=3)

    pos_vector = pos[:3]
    position_array.append(pos_vector)

area_covered_array_km2 = []

for index in range(len(position_array) - 1):    # "-1" to prevent index error
    r1 = position_array[index] # r1 --> distance from sun to earth at time t1
    r2 = position_array[index+1]# r2 --> distance from sun to earth at time t2

    #Area swept out = (1/2) * (|r1 X r2|)
    cross_product = np.linalg.norm(np.cross(r1, r2))
    # Magnitude calculation of r1 and r2 cross product

    area = 0.5 * cross_product
    area_covered_array_km2.append(area)

area_in_km2 = np.mean(area_covered_array_km2)
print(area_in_km2)      # Value comes to about 2.227 × 10¹³ km²

#Approximate difference in area
for index in range(len(area_covered_array_km2) - 1):
    diff = area_covered_array_km2[index] - area_covered_array_km2[index+1]
    print(diff)
    break
```

From the above code, it was observed that the mean value of the area swept per time interval was 2.227×10^{13} km^2.
The approximate difference in area with respect to each time frame was found to be 562061.7734375 km^2, which in comparison to the mean area value, can be considered to be negligible.

4. Plotting the data computed using the Matplotlib library

```
earth_orbit_array = []

for time_instnce in time_array_in_et:
    # Obtaining the position of the earth w.r.t the sun for each time instance
    earth_position, _ = spiceypy.spkgps(targ=3,
```

```
                                        et=time_instnce,
                                        ref="ECLIPJ2000",
                                        obs=10)

    earth_orbit_array.append(earth_position)

earth_orbit_array = np.array(earth_orbit_array)  # Conversion to a numpy.array

# Obtaining the Radii of the Sun for graph scaling purposes
_, radii_sun = spiceypy.bodvcd(bodyid=10, item="RADII", maxn=3)

# Array containing the normalized values of the distance from Earth to Sun
earth_orbit_array_norm = earth_orbit_array / radii_sun
```

```
earth_orbit_array_xy = earth_orbit_array_norm[:, 0:2]
# Extracting only the x and y coordinates for the two-dimensional plot

pyplt.style.use("dark_background")

fig, ax = pyplt.subplots(figsize=(8,8))     # Dimensions of the plot

ax.set_xlim(-400,400)
ax.set_ylim(-400,400)
ax.set_aspect("equal")
ax.grid(True, linestyle = "dotted", alpha = 0.5)

fig_sun = pyplt.Circle(xy=[0.0, 0.0],
                       radius=100,
                       color="Darkorange",
                       alpha = 0.6,
                       label="Sun")
ax.add_artist(fig_sun)     # Adding the Sun to the Plot

# Random value of index 1120 assumed to plot the Earth
fig_earth = pyplt.Circle(xy=[earth_orbit_array_xy[1120][0],
                             earth_orbit_array_xy[1120][1]],
                             radius = 25, color="blue",
                             alpha=1, label="Earth")
ax.add_artist(fig_earth)   # Adding the Earth to the Plot

# Plotting the motion of the Earth around the Sun using x and y coordinates
pyplt.plot(earth_orbit_array_xy[:, 0],
           earth_orbit_array_xy[:, 1],
           ls = "solid", color = "blue",
           alpha = 0.9)

#TIME FRAME 1 (TF-1)
num_lines_1 = 2
```

```
# Creating an evenly-spaced array of random values, with a step value of 2
indices_1 = np.linspace(100, 3160, num_lines_1, dtype=int)


for index in indices_1:
    x, y = earth_orbit_array_xy[index]
    ax.plot([0, x], [0, y], color="red", ls="dashed",
            alpha = 0.6, label="Area covered (TF-1)")
    #[0, x], [0, y] --> denotes the point of origin of the line

#TIME FRAME 2 (TF-2)
num_lines_2 = 2

#Step value (assumed): +3060 (taken as time interval)
indices_2 = np.linspace(1900, 4960, num_lines_2, dtype=int)


for index in indices_2:
    x, y = earth_orbit_array_xy[index]
    ax.plot([0, x], [0, y], color="pink", ls="dashed",
            alpha = 0.6, label="Area covered (TF-2)")

pyplt.title("Plot Visualization of Kepler's Second Law", color="Green",
            fontsize=18)
pyplt.xlabel("1 unit: 6963.4 km", fontsize=10, color="yellow")
pyplt.ylabel("1 unit: 6963.4 km", fontsize=10, color="yellow")

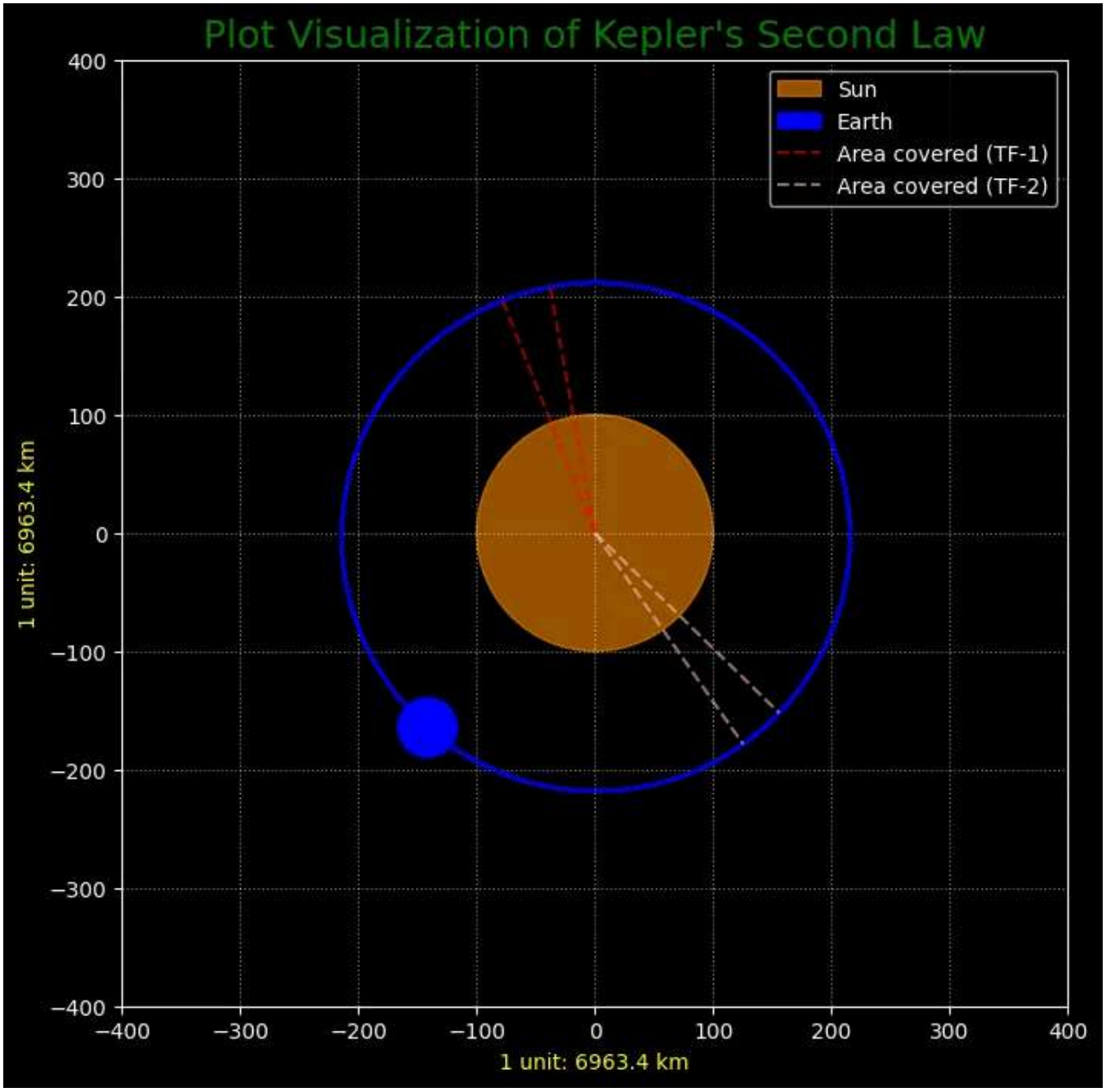
# Labels
handles, labels = ax.get_legend_handles_labels()

unique_labels = {}

for handle, label in zip(handles,labels):
    if label not in unique_labels:
        unique_labels[label] = handle

pyplt.legend(unique_labels.values(), unique_labels.keys())
```

With the above code computed, it establishes the below plot:



Note: The sizes of the Earth and Sun are not to scale. They have been exaggerated for visualization purposes.

To conclude, the article provides an outline on the verification of Kepler's Second Law of Planetary Motion, accompanied by a graph to showcase its better visualization.

Despite the variations of the Earth's speed as it orbits the sun, the area swept out by the Earth has seen to be nearly constant and is better visualized by noting its positions during perihelion and aphelion from the plot displayed.

Kepler | Astronomy | Data Analysis | NASA | Python